Final Report: Exploring Columnar Execution in DuckDB through a Sales Analytics Application

Yunhui Dong

1. Introduction & Motivation

The project examines the ways how the internal architecture of DuckDB can affect the operation of analytics applications. A simple sales analytics application is created for demonstration. The aim of the project is to establish the connection between the inner architecture of the database and the functionality provided by applications, such as aggregation, filtering and summarized reporting.

DuckDB was chosen because of its characteristics as an analytical database meant for OLAP queries. Unlike row-store databases that are designed to perform transactional processing, DuckDB is built to support scanning, filtering, and aggregation on large volumes of data. This makes it suitable for exploring the effect of database structures on reporting and analysis applications.

To make these ideas concrete, I have implemented a simple program for performing sales analytics using Python based on the supermarket sales data. Operations provided by this application include computing sales by city, computing the average sales by product category, filtering by date ranges, computing payment summaries, and computing summaries by branches and product categories. This approach provides a useful way of understanding the inner workings of DuckDB.

2. Database System Overview

DuckDB is an analytical database engine that has been designed specifically for use in online analytical processing (OLAP) and analytical processing tasks. DuckDB allows for SQL operations on structured tabular data. DuckDB is a relatively small engine and is easily integrated into Python, making it ideal for applications that require local analysis without a database server.

One of the main advantages of DuckDB is that it has been optimized for analytical queries like scans, filtering, projection, joining, and grouping. All these types of queries occur frequently when working with dashboards, reports, and summary-focused applications.

Unlike other systems optimized for transactional workloads, DuckDB is specifically built for heavy reading operations.

DuckDB is an ideal choice for this project since the workload revolves around performing aggregate and filter operations on a sales database. In this project, it will be important to study how the performance of these analytical queries is affected by the execution architecture of DuckDB.

3. Internal Architecture

The areas of focus for this project include storage, query execution, and to some extent indexing within DuckDB's architecture. However, the concepts of distribution and replication have not been included as part of this project since they are less relevant to DuckDB's architecture, which is not distributed.

3.1 Storage

Column storage is used by DuckDB. In column storage, data in each row of a particular column is stored together, whereas in row storage, the entire row is saved together. Columnar storage helps in optimizing analytical applications where not all columns may be required to execute queries.

In such a scenario, for instance, calculating total sales per city depends on data stored in the city and sales columns, but not on other columns such as gender, ratings, and payment unless they play a role in the execution of the process.

3.2 Query Execution

One of the main features used by DuckDB is vectorized query processing, where the operations are performed on batches of data instead of processing each row individually. This technique proves highly efficient when performing scan, filter, projection, and grouped aggregation. Considering that the application is heavily reliant on the performance of such queries, vectorization stands out as one of the most critical components affecting the application’s performance.

This technique becomes highly relevant for analytical purposes because, most often, the goal is to summarize rows, not to read or write individual records.

3.3 Indexing

Indexing will not be the main concern of this project. The tasks involved in the application can mostly be described as scan, filter, and grouping operations; there will not be any point operations nor index-based lookup tasks. The most important thing about this particular database implementation is the operation of column selection along with grouping operations, which use operators like the HASH_GROUP_BY operator. Therefore, indexing plays an inferior role in this case compared with storage and execution.

3.4 Distribution / Replication

DuckDB is a single node database. Hence, distribution and replication are not relevant in this project. Distribution and replication are concepts related to clustered databases. The project focuses more on analytics performed on the same node rather than the cluster.

4. Application Design

The application created for this work is a sales analytics application that uses Python and DuckDB. It does not aim at providing a sophisticated user interface, but at illustrating the relationship between analytical application functions and inner workings of databases.

4.1 Data Source and Schema

The data used for the project comes from a CSV file named “SuperMarket Analysis.csv.” It is imported into DuckDB as a relational table named supermarket_sales. The table includes fields such as:

- invoice ID
- branch
- city
- customer type
- gender
- product line
- quantity
- sales
- sale date
- payment method

- gross income
- rating

These fields are enough to perform multiple analytical queries using the grouping, filtering, and aggregation.

4.2 Application Workflow

Workflow of the Application:

1. Importing the CSV dataset in DuckDB.
2. Normalizing the column names so that they can be used for SQL queries.
3. Creating the Supermarket Sales table.
4. Running the analytics queries.
5. Analyzing the internal behavior through EXPLAIN and EXPLAIN ANALYZE.
6. Validating the behavior on scale datasets and calculating the average runtime.

4.3 Main Application Operations

The application supports several analytical operations, including:

- total sales by city
- average sales by product line
- total sales in January 2019
- transaction count and total sales by payment method
- total sales by branch and product line

All these functions have been included because they constitute a representative set of analytical tasks, with good examples of grouping and filtering.

4.4 User Interaction

The application was demonstrated through a notebook-based workflow and a lightweight UI layer. Interaction with the application takes place through the selection of operations, like sales by cities, date range filters, or product line aggregation. This process returns output tables along with the internal query execution plans.

5. Mapping Internals to Application Behavior

It is one of the main purposes of this project to explain how the inner workings of databases affect applications. For every critical application process, this report will explain what the application does, what the database does, and why what the database does matters.

5.1 Operation 1: Total Sales by City

On the application level, the task calculates the sales total for each city. This is a fairly standard business reporting query and an example of a standard aggregation task in a dashboard context.

Internally,DuckDB performs a sequential scan on the supermarket_sales table, processing only those columns of interest in the context of the query, namely the city and sales columns. In order to perform the aggregation, DuckDB uses the HASH_GROUP_BY operator to do a GROUP BY on city and calculate SUM(sales). Finally, the results are sorted by descending sales totals.

The point about this internal behavior of DuckDB is that analytical applications usually aggregate just a small number of dimensions or measures but not whole records. The ability of DuckDB to access only necessary fields and perform a grouped aggregation makes it suitable for such scenarios.

5.2 Operation 2: Total Sales in a Date Range

In terms of the application itself, the operation involves computing total sales over a certain period of time, for example, the month of January 2019. This is yet another common operation when it comes to analytics, as most reporting applications need to have some time-based summary information.

Internally, the task involves using DuckDB to filter out the rows by sale_date and then aggregating the sales data from those rows. The main important attributes for this particular problem include sale_date and sales.

The importance of this operation lies in the interaction between filtering and aggregation in analytics processing.

5.3 Operation 3: Average Sales by Product Line

In terms of the application-level process, this computation is based on the comparison of product lines through the average number of sales. It helps understand which product lines have the connection with higher average purchase values.

From an internal perspective, DuckDB focuses mainly on the product_line and sales columns. It executes the group_by on the product_line column and calculates the mean value for sales.

The reason why it is important is that it shows that multiple queries are used within the project. Various calculations and computations have internal processes similar to those described here.

5.4 Additional Operations

Both payment summary and branch-product summary add more diversity to the processing load. This is because these tasks show that the system can handle several business aspects rather than one simple task. Even though the particular groups will be different, the basic database ideas remain constant. These ideas include column projection, grouping aggregation, and analytical summaries.

6. Comparison with MySQL and MongoDB

In this part, DuckDB will be analyzed alongside MySQL and MongoDB in relation to architectural structure, strengths, weaknesses, and appropriate use cases.

6.1 DuckDB vs MySQL

MySQL is a widely-used relational database management system that is usually linked to Online Transaction Processing (OLTP) tasks, transactional tasks, and row orientation. This technology performs well with record-level inserts, updates, and deletes. Additionally, MySQL is good at processing concurrent application loads such as web backends.

Unlike MySQL, DuckDB is specifically tailored for OLAP tasks and analytical tasks. In the current project, the most frequent tasks include groupby, filter, and aggregate operations in a sales table. These tasks can be effectively processed by DuckDB since it allows one to perform scanning of relevant columns in analytical workloads.

In terms of strengths, DuckDB is more appropriate for analytical reporting, embedded analytics, and local data processing. On the contrary, MySQL is suitable for transactional processing, concurrent CRUD applications, and operational databases.

In terms of weaknesses, DuckDB does not have strong features for processing high concurrency transactions. Meanwhile, MySQL is not ideally suited to analytical processes because of its columnar nature.

6.2 DuckDB vs MongoDB

DuckDB is a relational database system built on SQL, making it ideal for dealing with tabular data such as the supermarket sales data used in this study. Its workload entails aggregation by group, filtration, and summarization of well-defined attributes, activities that fit perfectly into the duckdb's relational and analytics paradigm.

MongoDB is a non-relational database that can be described as document-oriented or NoSQL. It is usually the best choice where schema flexibility is needed or where documents similar to JSON need to be stored.

In terms of strengths, DuckDB is very effective in performing analytical and structured processing. MongoDB works better when flexible schemas and document-oriented models are involved.

In terms of weaknesses, DuckDB cannot work effectively where there is need for flexibility of schemas or document models. On the other hand, MongoDB does not work well where there is a need for columnar analytics as done in this study.

Therefore, DuckDB is the most appropriate choice for this application.

7. Limitations & Lessons Learned

The project has a few limitations. The first one is that the original database is relatively small, leading to inconsistent runtime variations. Another issue is that the current focus of the project is analytical queries as opposed to transactional queries, and the application user interface is relatively simple.

The main lesson learned throughout the project is that using DuckDB in Python is incredibly convenient. It was also noticed that EXPLAIN and EXPLAIN ANALYZE commands allow

understanding physical behavior behind queries, as well as the importance of increasing the database size to better observe workload behavior.

Future directions could be using a bigger dataset, investigating more types of queries, conducting comparison experiments with MySQL or MongoDB, and creating an improved interface.

8. Conclusion

This project explored how the internal structure of DuckDB helps the behavior of analytical applications within a sales analysis application. It shows that processes like group aggregation and date filtering are well suited to the columnar nature of DuckDB.
In summary, DuckDB proved highly suitable for the structured analytical workload, and this project showed how database internals can be explained using real-life operations.

9. Project Links

GitHub Repository: https://github.com/YunhuiDong/dsci551_spring2025_final_project

References

Raasveldt, Mark, and Hannes Mühleisen. “DuckDB.” *Proceedings of the 2019 International Conference on Management of Data*, 2019.

DuckDB Documentation. “Why DuckDB.”